Rust: Segurança na Programação

Conceitos Básicos

Nuno Neto · 2026

CONNECT5
EDIH European Digital Innovation Hubs Network
U.PORTO FACULDADE DE CIÊNCIAS UNIVERSIDADE DO PORTO
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
PRR Plano de Recuperação e Resiliência

Variáveis em Rust

Declaração

A keyword **let** declara uma variável. O tipo pode ser omitido — o compilador infere-o na maioria dos casos.

```
let nome: tipo = valor;
let valor = 0i32;
```

Naming Shadowing

É possível fazer **shadowing** — declarar uma nova variável com o mesmo nome.

```
let x: i32 = 5;
let x: i32 = x + 1; // shadowed
println!("{}", x); //Imprime o valor 6
```

Scoping em Rust

Em Rust, um **scope** é o intervalo para o qual uma variável é válida. Scopes são definidos por chaves {}. Quando um item sai de scope, ele é automaticamente **dropped** (libertado da memória).

O que define o Scope

Blocos delimitados por {} criam novos scopes. Variáveis declaradas dentro desses blocos só existem enquanto o execution pointer estiver dentro deles.

Lifetime e Dropping

Ao final de um scope, o Rust chama automaticamente o método drop, limpando a memória associada àquela variável.

Exemplos de Scope

Scope Básico e Destruição

```
{
    let s = String::from("hello");
    // s é válida aqui
} // s sai de escopo e é liberada
```

Shadowing em Nested Scopes

```
let x = 10;
{
    let x = 20; // Shadowing: esconde o x externo
    println!("Interno: {}", x);
}
// Aqui, o valor de x volta ao valor da segunda
declaração, pois a inner shadow deixa de estar em scope
println!("Externo: {}", x); // Imprime 10
```

A gestão de memória baseada em scopes é o pilar do sistema de **Ownership** do Rust, garantindo segurança sem a necessidade de um Garbage Collector.

Mutabilidade de Variáveis

Outras linguagens (C, Java...)

Variáveis são **mutáveis por defeito**.

Todas as variáveis são sujeitas a sofrer alterações a não ser que sejam especificamente anotadas. No caso do Java, são as variáveis **final**, em C as variáveis **const**, etc.

Rust

Variáveis são **imutáveis por defeito**. Para tornar uma variável mutável, usa-se a keyword **mut**.

```
let x = 1;
x = x + 1; // Não é permitido pois x não é mut

let mut y = 1;
y = y + 1; // Correto
```

Tipos de Dados

Todos os valores em Rust têm um tipo conhecido pelo compilador. Quando não é possível inferir, é obrigatória a anotação explícita.

Tipos Escalares

- Inteiros (**i8–i128**, **u8–u128**, **isize/usize**)
- Vírgula flutuante (**f32**, **f64** — IEEE-754)
- Booleanos (**bool**: true/false)
- Caracteres (**char** — Unicode Scalar Value, 4 bytes)

Tipos Compostos

- **Túpulos** — valores de tipos variados, tamanho fixo
- **Arrays** — valores do mesmo tipo, comprimento fixo

Tipos Escalares em Detalhe

Inteiros

12 tipos (6 tamanhos × signed/unsigned). **Signed (i)**: $-(2^{n-1})$ a $2^{n-1}-1$. **Unsigned (u)**: 0 a 2^n-1.

Vírgula Flutuante

Apenas **f32** e **f64**. Recomenda-se **f64** em sistemas modernos pela maior precisão. Usar **f32** quando a pegada de memória é crítica.

Bool & Char

bool: true/false. **char**: Unicode Scalar Value (u32 em memória) — suporta UTF-8, UTF-16, ASCII, ao contrário do **char** em C (apenas 1 byte).

Tipos Compostos

Túpulos

Agrupam valores de **tipos variados**. Tamanho fixo após declaração. Suportam desestruturação e **dot notation**.

```
let tupulo: (i32, i32) = (500, 100);
let (x, y) = tupulo;
let first = tupulo.0; // 500
```

Arrays

Guardam valores do **mesmo tipo**, comprimento fixo e conhecido pelo compilador. Úteis para dados na stack. Acessos em runtime são verificados pelo Rust para garantir segurança.

```
let array: [u32; 5] = [1, 2, 3, 4, 5];
let primeiro = array[0];
```

Funções

Declaradas com a keyword **fn**, seguida do nome, argumentos e return type (opcional para funções **void**). A anotação dos tipos dos argumentos é **obrigatória**.

```
fn test() {} // void
fn test_2() -> () {} // void explícito
fn test_3() -> i32 { 1 } // retorna i32
fn add(a: i32, b: i32) -> i32 { a + b }
```

Controlo de Fluxo

Condicionais

if, **else if**, **else** — branches dependentes de condições booleanas.

Loops

loop (while true)

while (condicional)

for (iteração sobre coleções).

Interrupção

break — termina o ciclo (pode extrair valores no caso de ser um **loop**).

continue — salta a iteração atual.

return — termina a função inteira.

break vs return em Loops

A distinção entre **break** e **return** é fundamental para controlar o fluxo de execução.

break

Interrompe o loop. A execução **continua** após o scope do loop. Pode extrair um valor: **break valor;**

```
let result = loop {
  let n: i32 = rand::random();
  if n == 0 {
   break n;
  }
};
```

continue

Salta apenas a iteração atual. O loop **não** é interrompido — prossegue para a próxima iteração.

```
loop {
  if index == 1 {
    continue;
} else if index == 2 {
    break;
}

println!("Index: {}", index);
};
```

return

Interrompe a **função inteira**, mesmo dentro de um loop. Ao contrário do break, nada após o return é executado.

```
if true { return 1; }
println!("Não é executado");
```